人工智慧無線通訊
Artificial Intelligence Wireless
HW-1.2

Exercise 1.5

Answer the following questions based on your understanding of RNNs and LSTM.

(a) What are the improvements of LSTM compared to ordinary RNNs?

- **Solves the vanishing gradient problem**
  LSTM introduces a memory cell and gating mechanisms that allow gradients to flow for longer sequences during backpropagation.
- **Captures long-term dependencies**
  Unlike standard RNNs that mainly learn short-term patterns, LSTM can retain information over long time steps.
- **Selective memory control**
  Through gates, LSTM can decide what information to keep, update, or discard, making it more stable and effective for sequence tasks.

(b) What is the function of the three gates in LSTM (input gate, forget gate, output gate)?

- **Input Gate**
  - Controls how much new information is written into the cell state.
  - Determines which parts of the current input should update the memory.
- **Forget Gate**
  - Decides which information from the previous cell state should be removed.
  - Helps discard irrelevant or outdated information.
- **Output Gate**
  - Determines which part of the cell state is used as the output (hidden state).
  - Controls what information is passed to the next time step or next layer.

(c) Why does the gradient vanishing problem occur when training RNNs? How does LSTM solve the gradient vanishing problem? Explain briefly.

When training RNNs using **Backpropagation Through Time**:

- Gradients are repeatedly multiplied by weights and activation derivatives.
- If these values are **less than 1**, gradients shrink exponentially.
- As the sequence becomes longer, gradients approach **zero**, making it difficult to update early layers.

This makes RNNs unable to learn **long-term dependencies**.

LSTM mitigates this problem by:

- **Using a cell state with nearly linear information flow**
  The cell state allows gradients to pass through many time steps with minimal decay.

- **Gate mechanisms**
  The **input, forget, and output gates** regulate information flow, preventing unnecessary gradient shrinking.
- **Constant error flow**
  The design enables gradients to propagate through time more effectively.

As a result, **LSTM can learn long-range dependencies that standard RNNs cannot**.

Exercise 1.6

When working with sequential data or natural language tasks, would you choose to use an attention mechanism or a CNN? What are their respective strengths and weaknesses?

Attention Mechanism

Strengths

1. Captures long-range dependencies:
   Attention can directly model relationships between any two tokens in a sequence, even if they are far apart.
2. Global context awareness:
   Each token can attend to all other tokens, allowing the model to understand the whole sequence.
3. Flexible sequence modeling:
   Widely used in Transformers, which perform very well in NLP tasks such as: machine translation, text generation, question answering.

Weaknesses

1. High computational cost:
   Standard attention has $O(n^2)$ complexity with respect to sequence length.
2. Memory intensive:
   Long sequences require large GPU memory.
3. May require large datasets:
   Transformer-based models often need large-scale training data.

Convolutional Neural Networks (CNN)

Strengths

1. Efficient computation:
   Convolution operations are highly optimized and computationally efficient.
2. Good at capturing local patterns:
   CNNs work well for local feature extraction, such as n-gram features in text.
3. Lower memory cost:
   Compared with attention for long sequences.

Weaknesses

1. Limited receptive field:
   CNNs mainly capture local dependencies. Long-range relationships require many stacked layers.

2. Less flexible for sequence relationships:
   Information from distant tokens is harder to integrate.
3. Fixed kernel size limitation:
   Context window is constrained by filter size.

Exercise 1.7

(a) Answer the following questions based on your understanding of score-based generative models.

Similarities

- Both are **generative models** that learn the data distribution $P_{data}(x)$.
- Both use **neural networks**.
- Both can generate complex data such as images or audio.

Differences

	GAN	Score-Based Model
Main idea	Adversarial training (Generator vs Discriminator)	Learn the score $\nabla_x \log p(x)$
Training	Minimax game	Score matching
Sampling	One forward pass	Iterative generation
Stability	Can suffer from mode collapse	more stable

(b) Derivation of the Score

The **score function** is defined as:

$$s(x) = \nabla_x \log p(x)$$

Using the derivative of the logarithm:

$$\nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)}$$

Thus,

$$s(x) = \frac{\nabla_x p(x)}{p(x)}$$

This represents the direction in which the probability density increases the fastest.

(c) Equivalence of the Loss

Add Gaussian noise:

$$\tilde{x} \sim \mathcal{N}(x, \sigma^2 I)$$

The score of the conditional distribution is:

$$\nabla_{\tilde{x}} \log p(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2}$$

Therefore, the training loss becomes:

$$\mathbb{E}\left[\left\|s_\theta(\tilde{x}) + \frac{\tilde{x} - x}{\sigma^2}\right\|^2\right]$$

This is called denoising score matching, which is equivalent to the original score matching objective.

Exercise 1.8

Greedy Play supposes that the reinforcement learning player was greedy, that is, it always played the move that brought it to the position that it rated the best. Might it learn to play better, or worse, than a nongreedy player? What problems might occur?

Greedy play means the reinforcement learning agent always chooses the action with the highest current estimated value. It will usually learn worse than a non-greedy player. A greedy player does not explore other actions, so it may miss better strategies.

Problems that may occur

- **Lack of exploration**
  The agent never tries other actions, so it cannot discover potentially better moves.
- **Local optimum**
  The agent may get stuck with a strategy that seems best early but is not actually optimal.
- **Incorrect value estimates**
  Early estimates may be inaccurate, but greedy behavior keeps reinforcing them.
- **Poor long-term learning**
  Without exploration, the agent cannot improve its policy effectively.

Exercise 1.9

Answer the following questions:

(a) What is the difference between on-policy algorithms and off-policy algorithms in reinforcement learning?

On-policy

Learns using data generated by the same policy it is currently updating.

Example: PPO, TRPO, REINFORCE

Off-policy

Learns using data generated by another policy.

Example: DQN, DDPG, SAC, TD3

(b) What is the difference between value-based algorithms and policy-based algorithms in reinforcement learning?

Value-based

- Learns a value function $Q(s,a)$
- The policy is derived by choosing the action with the highest value.
- Example: Q-learning, DQN

Policy-based

- Directly learns a policy $\pi(a \mid s)$.
- Example: REINFORCE, PPO.

(c) What is the difference between deterministic methods and stochastic methods in reinforcement learning?

Deterministic policy

- The policy outputs one specific action for a state $a = \pi(s)$
- Example: DDPG, TD3.

Stochastic policy

- The policy outputs a probability distribution over actions $a \sim \pi(a \mid s)$
- Example: PPO, TRPO, SAC

(d) Determine the categories of reinforcement learning algorithms below. For example, DQN [40] is a valuebased off-policy deterministic reinforcement learning algorithm.

Algorithm	Type
REINFORCE	Policy-based, On-policy, Stochastic
Deep deterministic policy gradient (DDPG)	Policy-based, Off-policy, Deterministic
Trust-region policy optimization (TRPO)	Policy-based, On-policy, Stochastic
PPO	Policy-based, On-policy, Stochastic
Soft actor-critic (SAC)	Policy-based, Off-policy, Stochastic
Twin-delayed DDPG (TD3)	Policy-based, Off-policy, Deterministic

Exercise 1.10

Consider a continuing task without the terminal state. The agent operates in an environment with a discount factor $\gamma \in (0,1)$.

(a) Are the signs of these rewards important, or are only the intervals between them essential? Prove that adding a constant b to all the rewards will add a constant v_b to the value functions of all states and thus does not affect the comparisons between any states under any policies. What is v_b in terms of b and γ ?

If a constant b is added to every reward:

$$R'_t = R_t + b$$

The new value function becomes:

$$v'(s) = v(s) + \frac{b}{1-\gamma}$$

The value function is

$$v(s) = E\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}\right]$$

After adding b:

$$v'(s) = E\left[\sum_{k=0}^{\infty} \gamma^k (R_{t+k+1} + b)\right] = v(s) + \sum_{k=0}^{\infty} \gamma^k b$$

Since

$$\sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}$$

we get

$$v'(s) = v(s) + \frac{b}{1-\gamma}$$

Adding a constant shifts all state values by the same amount, so state comparisons do not change.

(b) Now consider adding a constant *b* to all the rewards in an episodic task, such as maze navigation. Would this have any effect on the value function of each state, or would it leave the task unchanged as in the continuing task above? Why or why not? Give an example.

Adding constant b does affect the value function.

Reason:

Episodes have different lengths, so the total added reward becomes different.

Example:

Two paths to the goal:

- Path A: 5 steps
- Path B: 10 steps

If we add b each step:

- Path A adds 5 b
- Path B adds 10 b

Thus, longer paths may appear more valuable, changing the policy.

Exercise 1.11

Why is Q-learning considered an off-policy control method? Supposing action selection is greedy; is Q-learning then exactly the same algorithm as SARSA? Will they make exactly the same action selections and action-value function updates?

1. Why is Q-learning considered an off-policy method?

Q-learning is off-policy because it learns the value of the optimal greedy policy, even if the agent follows a different behavior policy.

Update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_a Q(s',a) - Q(s,a)\right]$$

The update uses **$\max_a$ *Q(s',a)*** (the best possible action), not the action actually taken.

Therefore:

- Behavior policy: used to select actions (e.g., ε-greedy).
- Target policy: greedy policy used in the update.

Thus Q-learning is off-policy.

2. If action selection is greedy, is Q-learning the same as SARSA?

No, they are not exactly the same.

SARSA update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma Q(s',a') - Q(s,a)\right]$$

Here the update uses **the actual next action *a'*.**

3. Will they make the same action selections and updates?

Action selection

- If both use pure greedy policy, they may select the same actions.

Updates

The updates are still different:

- **Q-learning uses the maximum Q-value.**
- **SARSA uses the Q-value of the chosen action.**

Therefore, the updates are not identical.

Exercise 1.12

MountainCar-vo is a classic reinforcement learning task in the OpenAI Gym library. In this task, an underpowered car must drive up a mountain to reach the goal at the top. The car cannot reach the goal by driving directly up the hill due to insufficient engine power. Instead, it must build momentum by going back and forth between two hills. The state space includes the car's position and velocity, and the action space consists of three actions: push left, push right, or do nothing. The task is considered solved when the agent reaches the flag on top of the hill within a certain number of steps.

1. Neural Network Architecture

The original Actor and Critic networks had only one hidden layer.

The architecture was expanded to two hidden layers with Layer Normalization, which improves feature representation and stabilizes training.

Original

- state → Linear(128) → ReLU → Linear → output

Modified

- state → Linear(128) → ReLU → LayerNorm → Linear(128) → ReLU → Linear → output

2. Reward Shaping

The MountainCar environment has sparse rewards (−1 per step), making learning difficult.
To improve exploration, additional rewards were introduced:

- **Progress reward**: encourages movement toward the goal
- **Velocity reward**: encourages the car to build momentum
- **Goal bonus**: large reward when reaching the target position

This helps the agent learn the required back-and-forth momentum strategy.

3. PPO Training Improvements

Several techniques were added to stabilize training:

- Entropy bonus to encourage exploration
- Advantage normalization to reduce variance
- Bootstrap value estimation using the critic network

These modifications improve policy update stability.

The best result:

```
Training Finished
Best Episode : 932
Minimum Step : 83
end
```